

O S

操作系统



O S

Chapter 4

处理机调度



O S

1

分级调度

2

作业调度

3

进程调度

4

调度算法及算法评价

5

实时系统调度算法



作业

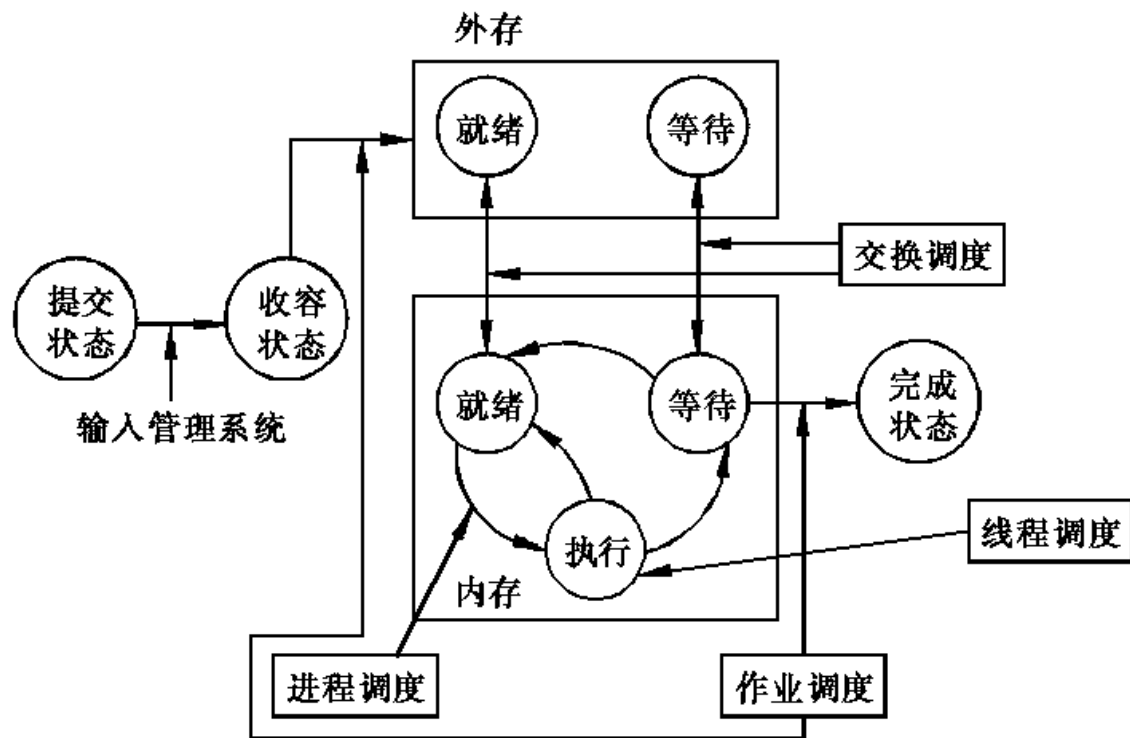
➤ 回顾：作业的概念

在一次应用业务处理过程中，从输入开始到输出结束，用户要求计算机所做的有关该次业务处理的全部工作称为一个作业。它包括作业的提交、执行和输出等过程。

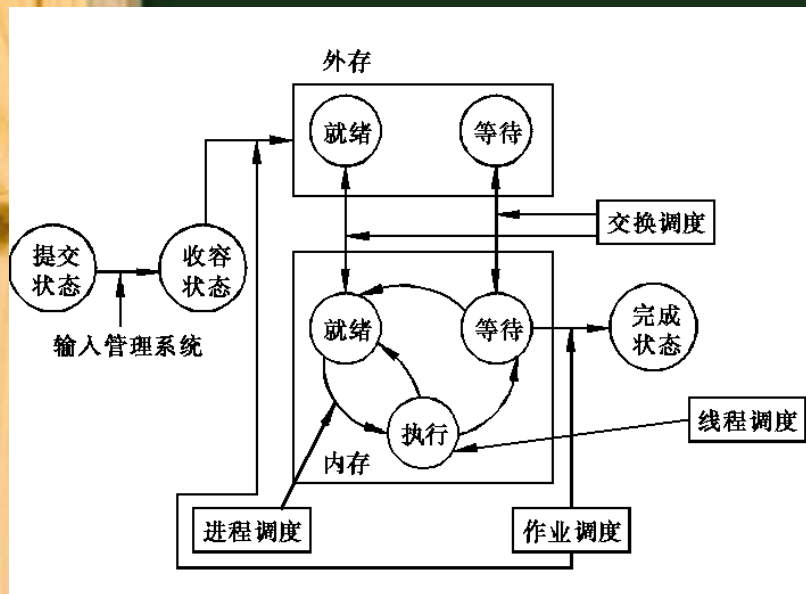
作业的状态及其转换

➤一个作业从用户提交开始到真正占有处理机而被执行，则要由系统经过**多级调度**才能实现（在有些系统，例如分时系统中，也可以由单级调度实现）。

作业的状态及其转换

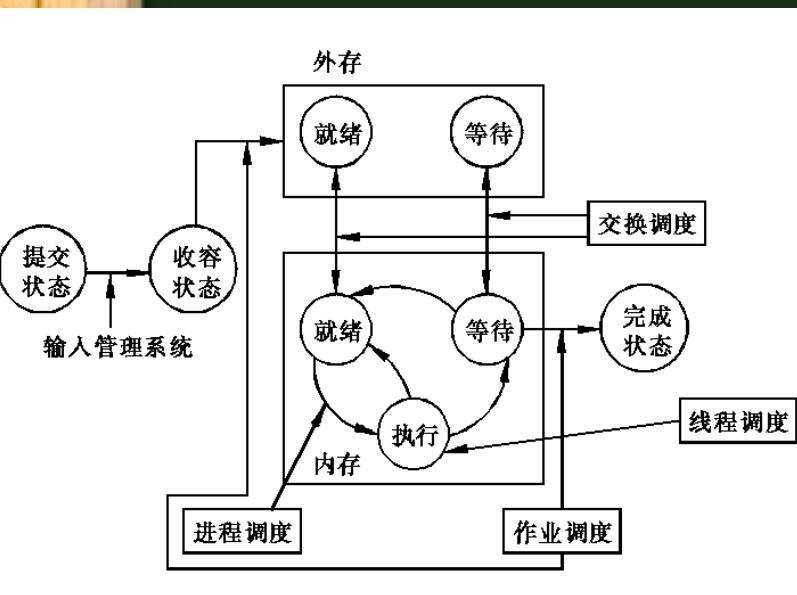


作业的状态及其转换



➤在进程有资格竞争处理机之前，**作业调度程序**必须先调用**存储管理、外设管理程序**，并按一定的选择顺序和策略从输入井中选择出几个处于后备状态的作业，为它们分配内存等资源和创建进程，使它们获得竞争处理机的资格。

作业的状态及其转换



➤ 为了提高资源的利用率，在有些操作系统中把一部分在内存中处于就绪状态或等待状态而在短时期内又得不到执行的进程、作业**换出内存**，以让其他作业的进程竞争处理机。这样，在外存中，除了处于后备状态的作业外，还存在有处于就绪状态而等待得到内存的作业。这就需要有一定的方法和策略为这部分作业分配空间。

调度的层次

➤ 只有那些参与竞争处理机所必需的资源都已得到满足的进程才能享有竞争处理机的资格。这时，它们处于内存就绪状态。这些必需的资源包括内存、外设及有关数据结构等。



调度的层次

作业
调度

交换
调度

进程
调度

线程
调度

➤ 又称宏观调度或高级调度。其主要任务是按一定的原则对外存输入井上的大量后备作业进行选择，给选出的作业**分配内存、输入输出设备**等必要的资源，并建立相应的进程，以使该作业的进程获得竞争处理机的权利。另外，**当该作业执行完毕时，还负责回收系统资源。**

调度的层次

作业
调度

交换
调度

进程
调度

线程
调度

➤称中级调度。其主要任务是按照给定的原则和策略，将处于外存交换区中的就绪状态或就绪等待状态的进程调入内存，或把处于内存就绪状态或内存等待状态的进程交换到外存交换区。**交换调度**主要涉及到内存管理与扩充。

调度的层次

作业
调度

交换
调度

进程
调度

线程
调度

➤ 又称微观调度或低级调度。其主要任务是按照某种策略和方法选取一个处于就绪状态的进程占用处理机。在确定了占用处理机的进程后，系统必须进行进程上下文切换以建立与占用处理机进程相适应的执行环境。

调度的层次

➤在多道批处理系统中，存在着作业调度和进程调度。但是，在分时系统和实时系统中，一般不存在作业调度，而只有进程调度、交换调度和线程调度。

调度的层次

➤这是因为在分时系统和实时系统中，为了缩短响应时间或为了满足用户需求的截止时间，作业不是建立在外存，而是直接建立在内存中。在这些系统中，一旦用户和系统的交互开始，用户马上要进行控制。因而，这些系统中没有作业提交状态和后备状态。它们的输入信息经过终端缓冲区为系统所接收，或者立即处理，或者经交换调度暂存外存中。



作业与进程的关系

➤首先，系统必须为一个作业创建一个**根进程**。然后，在执行作业控制语句时，根据任务要求，系统或根进程为其创建相应的**子进程**，然后，为各子进程分配资源和调度各子进程执行以完成作业要求的任务

进程是计算机为了完成用户

进程
1



进程
2



进程
n

任务实体而设置的执行实体

作业：用户向计算机提交任务的实体

O S

1

分级调度

2

作业调度

3

进程调度

4

调度算法及算法评价

5

实时系统调度算法



作业调度的功能

作业执行结
束时做善后
处理工作

记录系统中
各作业的状况

为被选中作
业做好执行
前的准备工
作

从后备队列
中挑选出一
部分作业投
入执行

作业名

作业类型

资源要求

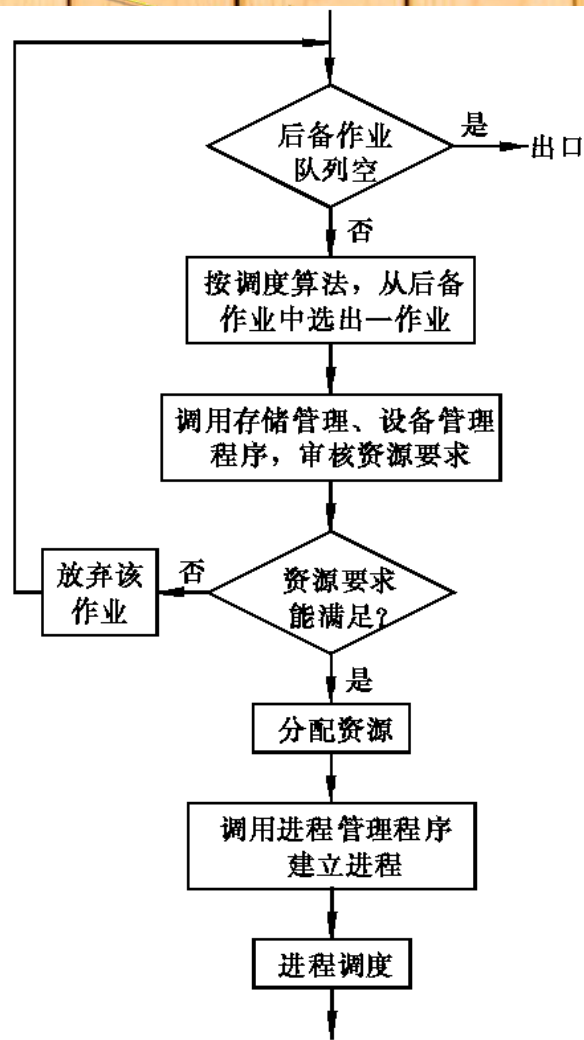
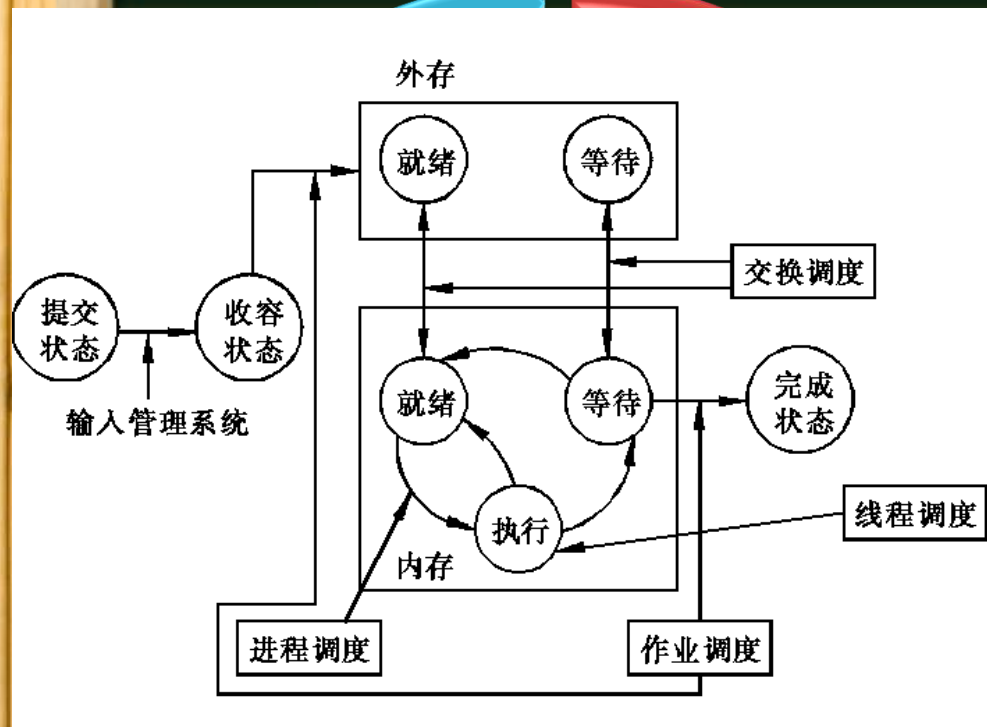
资源使用情况

优先级(数)

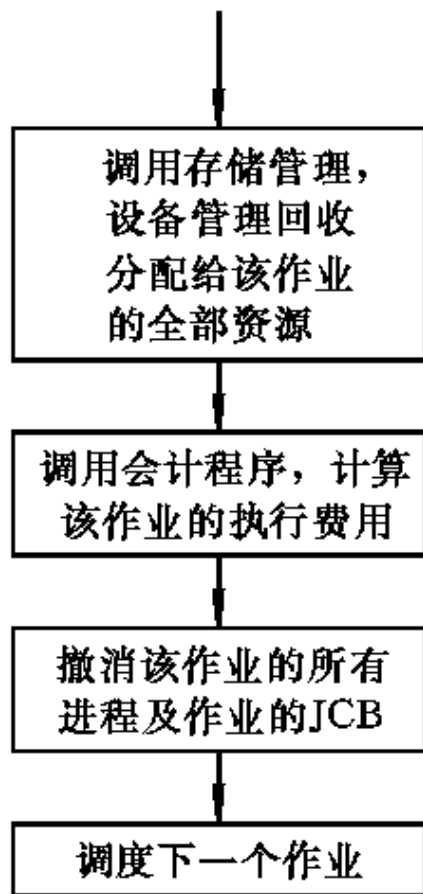
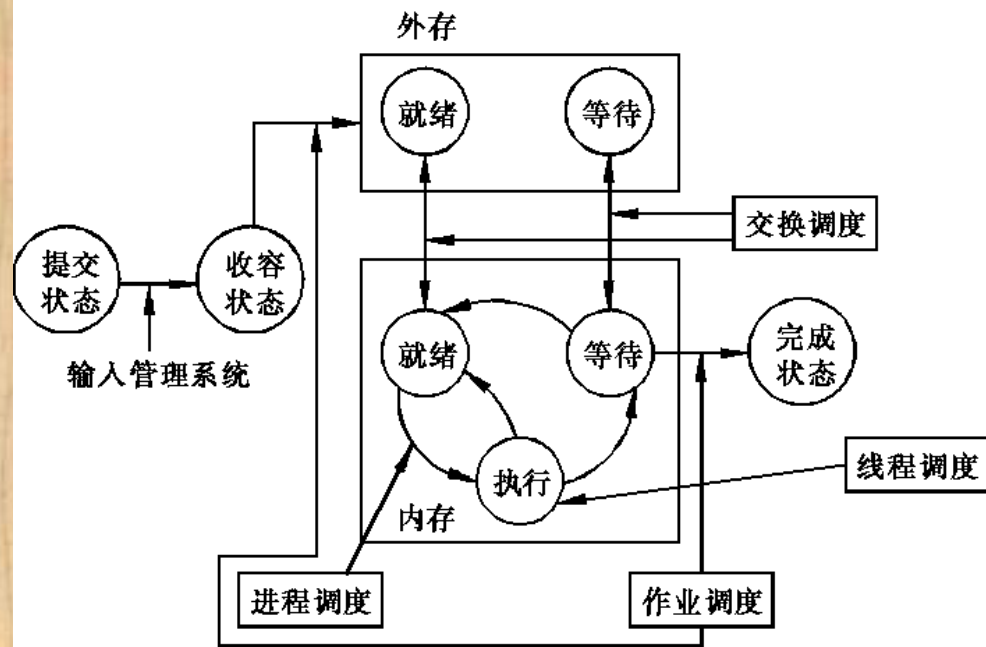
当前状态

其他

作业调度的功能



作业调度的功能



作业调度的目标

有快的响应
时间

对所有作业
应该是公平
合理的

每天执行尽
可能多的作
业；

应使设备有
高的利用率

➤由于这些目标的相互冲突，任一调度算法要想同时满足上述目标是不可能的。

➤如果考虑的因素过多，调度算法就会变得非常复杂。其结果是系统开销增加，资源利用率下降。

作业调度的目标

- 大多数操作系统都根据用户需要，采用兼顾某些目标的简单调度算法。
- 不同的系统对调度目标的要求不尽相同。

作业调度的目标

- 批处理系统：主要用于计算，对于作业的周转时间要求较高。
- 作业的平均周转时间或平均带权周转时间，被作为衡量调度算法优劣的标准。

作业调度的目标—批处理

T_{ei} 为作业*i*的完成时间

T_{si} 为作业的提交时间

➤ 作业*i*的周转时间 $T_i = T_{ei} - T_{si}$

➤ 对于被测定作业流所含有的 n ($n \geq 1$) 个作业来说, 其平均周转时间为:

$$T = \frac{1}{n} \sum_{i=1}^n T_i$$

➤ 一个作业的周转时间, 包含两部分: T_{wi} 主要指作业*i*由后备状态到执行状态的等待时间, 它不包括作业进入执行状态后的等待时间。

即: $T_i = T_{wi} + T_{ri}$

作业调度的目标—批处理系统

➤ 作业的 **带权周转时间**：是作业**周转时间**与作业**执行时间**的比： $W_i = T_i / T_{ri}$

➤ 对于被测定作业流所含有的几个作业来说，其**平均带权周转时间**为：

$$W = \frac{1}{n} \sum_{i=1}^n W_i$$

分时、实时系统

➤分时系统和实时系统，除了要保证系统吞吐量大、资源利用率高之外，还应保证有用户能够容忍的响应时间。因此采用**平均响应时间**被作为衡量调度策略优劣的标准。

O S

1

分级调度

2

作业调度

3

进程调度

4

调度算法及算法评价

5

实时系统调度算法



进程调度的功能

- 将系统中各进程的执行情况 and 状态特征记录在各进程的PCB表中。并且，进程管理模式根据各进程的状态特征和资源需求，将各进程的PCB表排成相应的队列并进行动态队列转接。进程调度模块通过PCB变化来掌握系统中所有进程的执行情况和状态特征，并在适当的时机从就绪队列中选择一个进程占据处理机。

记录系统中所有进程的执行情况

1

- 按照一定的策略选择一个处于就绪状态的进程，使其获得处理机执行。根据不同的系统设计目的，有各种各样的选择策略

选择占有处理机的进程

2

- 当进行上下文切换时，系统要首先检查是否允许做上下文切换。

- 然后，系统要保留有关被切换进程的足够信息，以便以后切换回该进程时，顺利恢复该进程的执行。

- 调度程序选择一个新的处于就绪状态的进程，并装配该进程的上下文，使CPU的控制权转换到被选中进程中

进行进程上下文切换

3

进程调度的时机

➤ 如果CPU执行方式为**不可剥夺方式**

正在执行的进程执行完毕

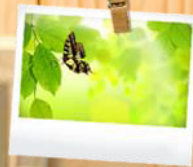
执行中进程自己调用阻塞原语将自己阻塞起来进入睡眠等待状态

执行中进程调用了P原语操作，从而因资源不足而被阻塞；
或调用了V原语操作激活了等待资源的进程队列

执行中进程提出I / O请求后被阻塞

在分时系统中时间片已经用完

在执行完系统调用，在系统程序返回用户进程时，可认为系统进程执行完毕，从而可调度选择一新的用户进程执行



进程调度的时机

➤ 如果CPU执行方式为**非不可剥夺方式**

就绪队列中的某进程的优先级变得高于当前执行进程的优先级，从而也将引发进程调度

进程上下文切换

- 进程上下文由正文段、数据段、硬件寄存器的内容以及有关数据结构等组成。
- 硬件寄存器主要包括存放CPU将要执行的下条指令虚拟地址的程序计数器PC，指出机器与进程相关联的硬件状态的处理机状态寄存器PSW，存放过程调用（或系统调用）时所传递参数的通用寄存器R以及堆栈指针寄存器S等。

进程上下文切换

➤ 数据结构则包括PCB等在内的所有与执行该进程有关的管理和控制用表格、数组、链等。在发生进程调度时，系统要做进程上下文切换。

进程上下文切换的步骤

(1) **决定是否做上下文切换以及是否允许做上下文切换。** 包括对进程调度原因的检查分析，以及当前执行进程的资格和CPU执行方式的检查等。

(2) **保存当前执行进程的上下文。** 这里所说的当前执行进程，实际上是指调用上下文切换程序之前的执行进程。如果上下文切换程序不是被那个当前执行进程所调用，且不属于该进程，则所保存的上下文应是先前执行进程的上下文，或称为“老”进程上下文。

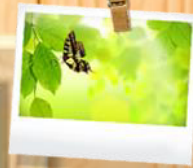
进程上下文切换的步骤

(3) 使用4.4节中所述进程调度算法，选择一个处于就绪状态进程。

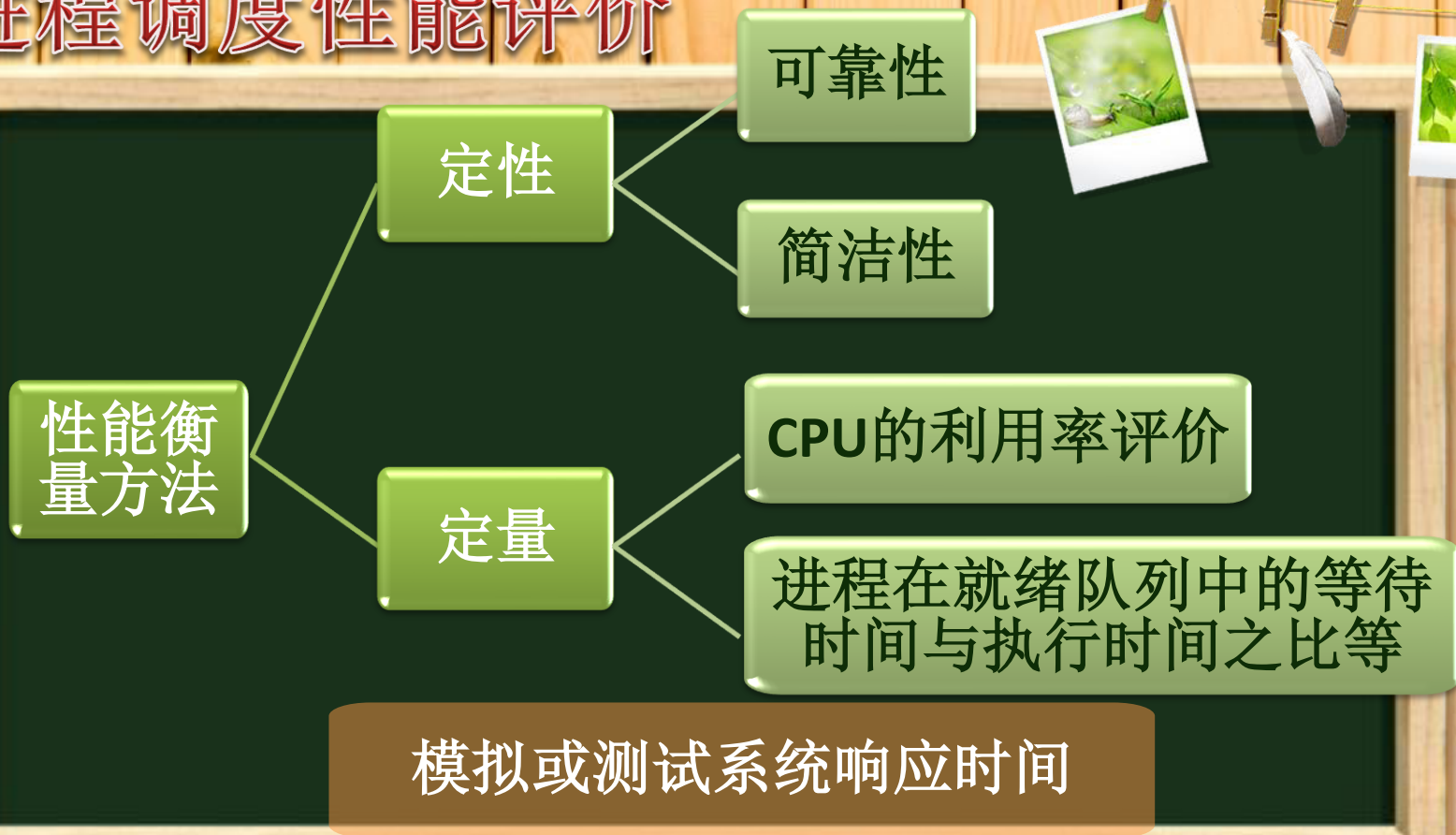
(4) 恢复或装配所选进程的上下文，将CPU控制权交给所选进程。

进程调度性能评价

➤ 进程等待时间的多少是要依靠进程调度策略和等待事件何时发生来决定的。因此，进程调度性能的衡量是操作系统设计的一个重要指标



进程调度性能评价



O S

1

分级调度

2

作业调度

3

进程调度

4

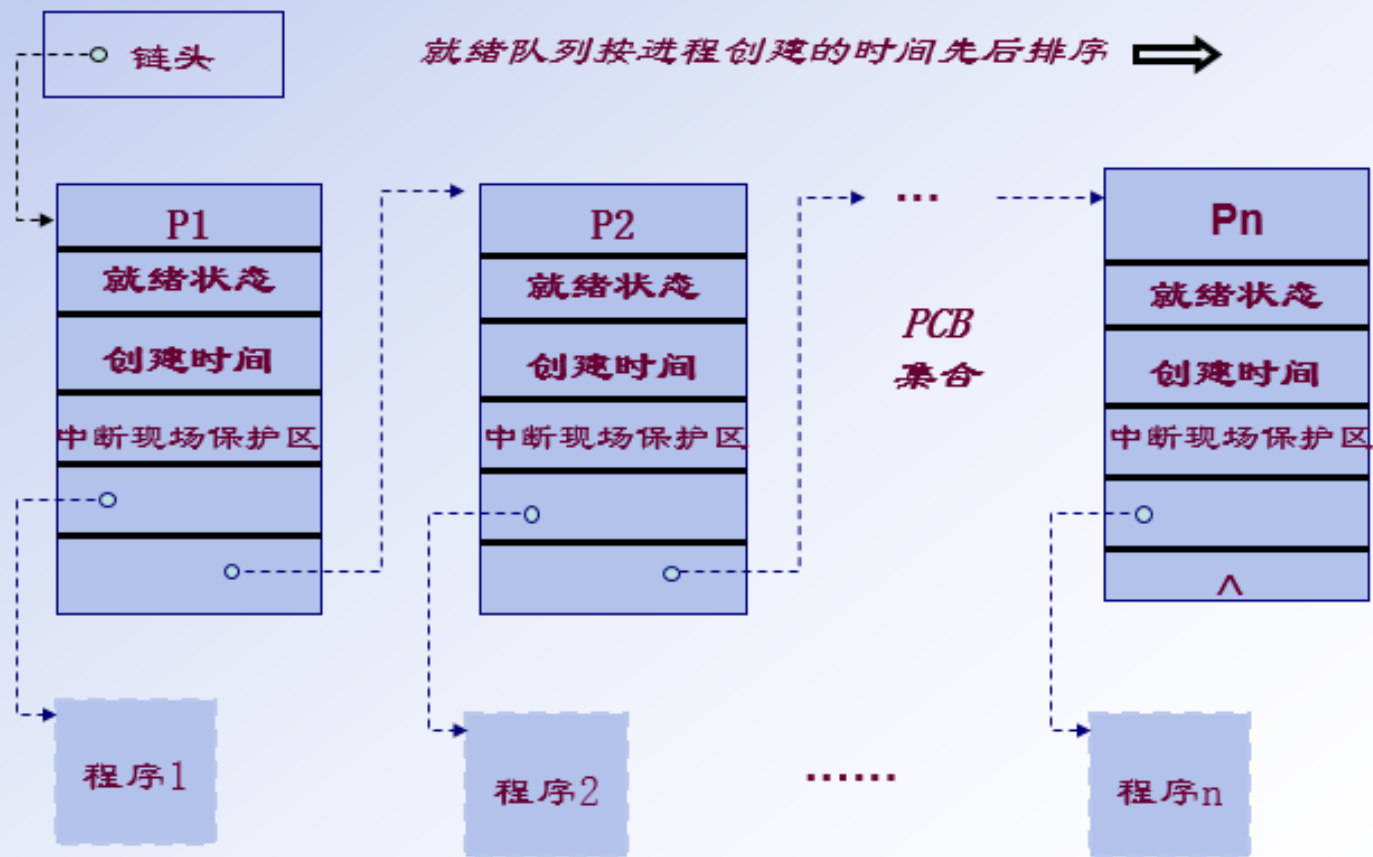
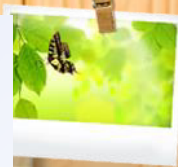
调度算法及算法评价

5

实时系统调度算法



先来先服务 (FCFS) 调度算法



先来先服务（FCFS）调度算法

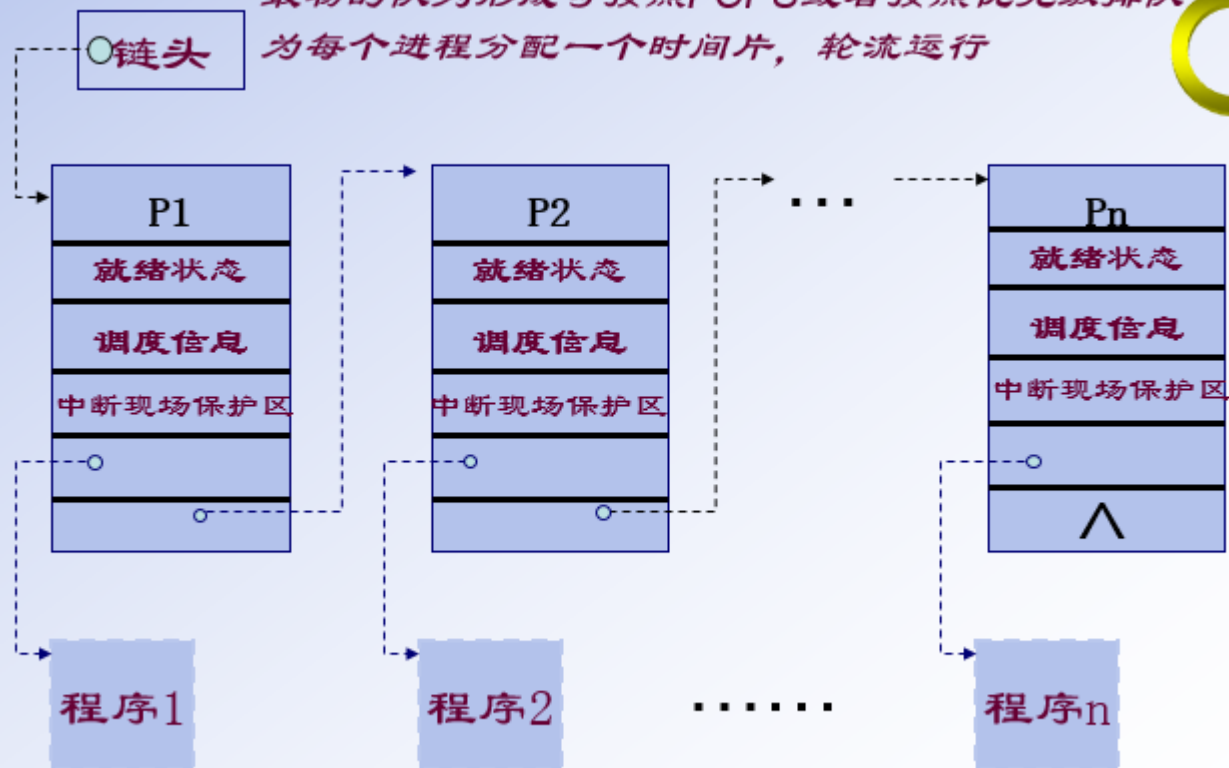
- 在没有特殊理由要优先调度某类作业或进程时，从处理的角度来看，FCFS方式是一种最合适的方法，无论是追加还是取出一个队列元素在操作上都是最简单的。
- 直观看，该算法在一般意义下是公平的。即每个作业或进程都按照它们在队列中等待时间长短来决定它们是否优先享受服务。不过对于那些执行时间较短的作业或进程来说，如果它们在某些执行时间很长的作业或进程之后到达，则它们将等待很长时间。

先来先服务（FCFS）调度算法

➤在实际操作系统中，尽管很少单独使用FCFS算法，但和其他一些算法配合起来，FCFS算法还是使用得相当多的。例如基于优先级的调度算法就是对具有同样优先级的作业或进程采用的FCFS方式。

轮转法 (RR) 调度算法

最初的队列形成可按照FCFS或者按照优先级排队
为每个进程分配一个时间片，轮流运行



轮转法 (RR) 调度算法

- 轮转法只能用来调度分配那些可以抢占的资源。将它们随时剥夺再分配给别的进程。CPU是可抢占资源的一种。但如打印机等资源是不可抢占的。

轮转法（RR）调度算法

➤ 作业调度是否可以采用轮转法？



➤ 作业调度是对除了CPU之外的所有系统硬件资源的分配，其中包含有不可抢占资源，所以作业调度不使用轮转法。

轮转法（RR）调度算法

- 在轮转法中，时间片长度的选取非常重要。
- 首先，时间片长度的选择会直接影响系统开销和响应时间。如果时间片长度过短，则调度程序剥夺处理机的次数增多。这将使进程上下文切换次数也大大增加，从而加重系统开销。反过来，如果时间片长度选择过长，比方说一个时间片能保证就绪队列中所需执行时间最长的进程能执行完毕，则轮转法变成了先来先服务法。



轮转法 (RR) 调度算法

➤ 时间片长度的选择是根据系统对响应时间的要求 R 和就绪队列中所允许的最大进程数 $N_{\max}$ 确定的。它可表示为: $q=R/N_{\max}$

➤ 若 q 为常数, 则当就绪队列中的进程数发生远小于 $N_{\max}$ 的变化, 则响应时间 R 看上去会大大减小。但是, 就系统开销来说, 由于 q 值固定, 从而进程上下文切换的时机不变, 系统开销也不变。

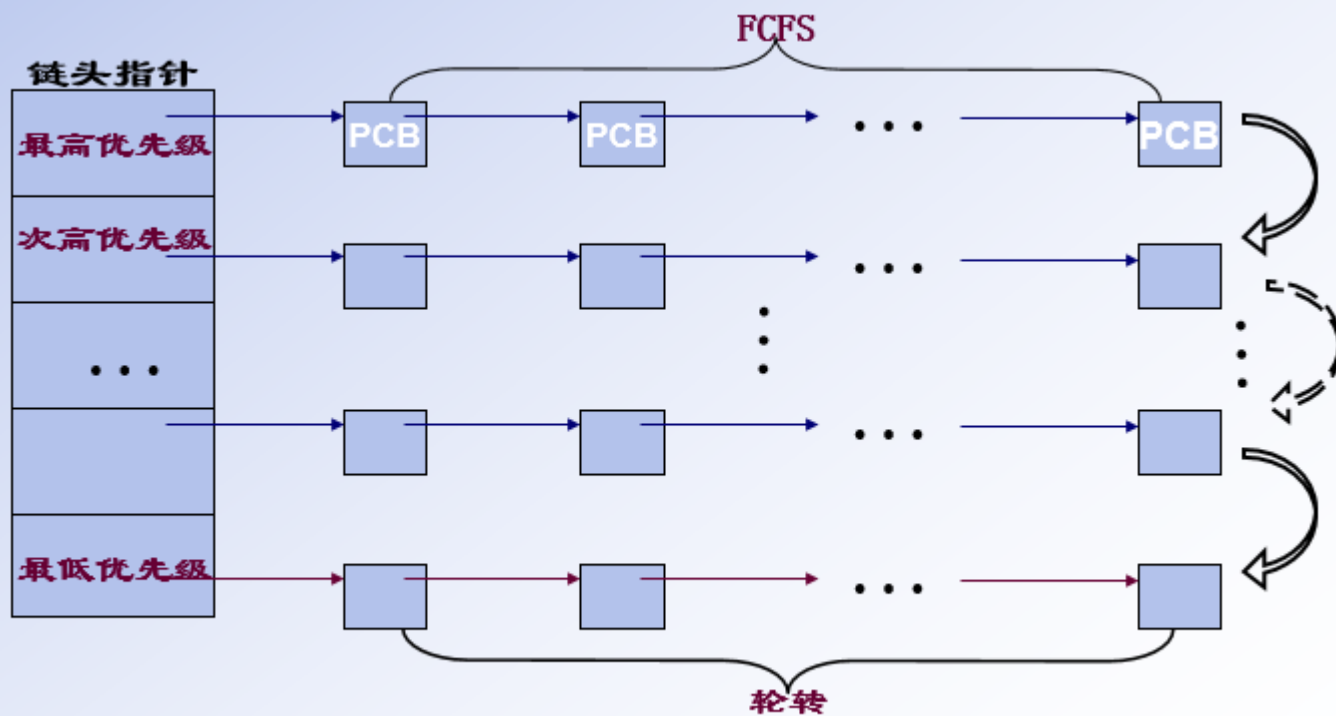
轮转法 (RR) 调度算法

➤ 更好的解决方法：每当一轮调度开始时，系统便根据就绪队列中已有进程数目计算一次 q 值，作为新一轮调度的时间片。这种方法得到的时间片随就绪队列中的进程数变化。

多级反馈轮转法（RRF）调度算法

- 分析：在轮转法中，加入到就绪队列进程的三种情况
 - 分给它的**时间片用完**，但进程还未完成，回到就绪队列的末尾等待下次调度去继续执行。
 - 分给该进程的时间片并未用完，只是因为请求I / O或由于进程的互斥与同步关系而**被阻塞**。当阻塞解除之后再回到就绪队列。
 - 新创建进程**进入就绪队列。

多级反馈轮转法 (RRF) 调度算法



优先级法调度算法

- 系统或用户按某种原则为作业或进程**指定一个优先级**来表示该作业或进程所享有的调度优先权。
- 该算法的**核心是确定**进程或作业的**优先级**。

优先级法调度算法

静态法

➤ 静态法根据作业或进程的静态特性，在作业或进程开始执行之前就确定它们的优先级，一旦开始执行之后就不能改变。

动态法

➤ 它把作业或进程的静态特性和动态特性结合起来确定作业或进程的优先级，随着作业或进程的执行过程，其优先级不断变化

优先级法调度算法---作业静态优先级

- (1) 由用户自己根据作业的紧急程度输入一个适当的优先级。为防止各用户都将自己的作业冠以高优先级，系统应对高优先级用户收取较高的费用。
- (2) 由系统或操作员根据作业类型指定优先级。作业类型一般由用户约定或由操作员指定。例如：可将作业分为：I / O繁忙的作业，CPU繁忙的作业，I / O与CPU均衡的作业，一般作业，等等。

优先级法调度算法---作业静态优先级

➤ (3) 系统根据作业要求资源情况确定优先级。例如根据估计所需处理机时间、内存量大小、I / O设备类型及数量等，确定作业的优先级。

可以根据其所要完成的功能划分为不同的类型，例如，**调度进程**、**I / O进程**、**中断处理进程**、**存储管理进程**等。这些进程还可进一步划分为不同类型和赋予不同的优先级。例如，在操作系统中，对于**键盘中断**的处理优先级和对于**电源掉电中断**的处理优先级是不相同的。

静态优先级

优先级。例如，在

有些系统中，进程被划分为**系统进程**和**用户进程**。

系统进程享有比**用户进程**高的优先级。

➤ (2) **将作业的静态优先级作为它所属进程的优先级。**

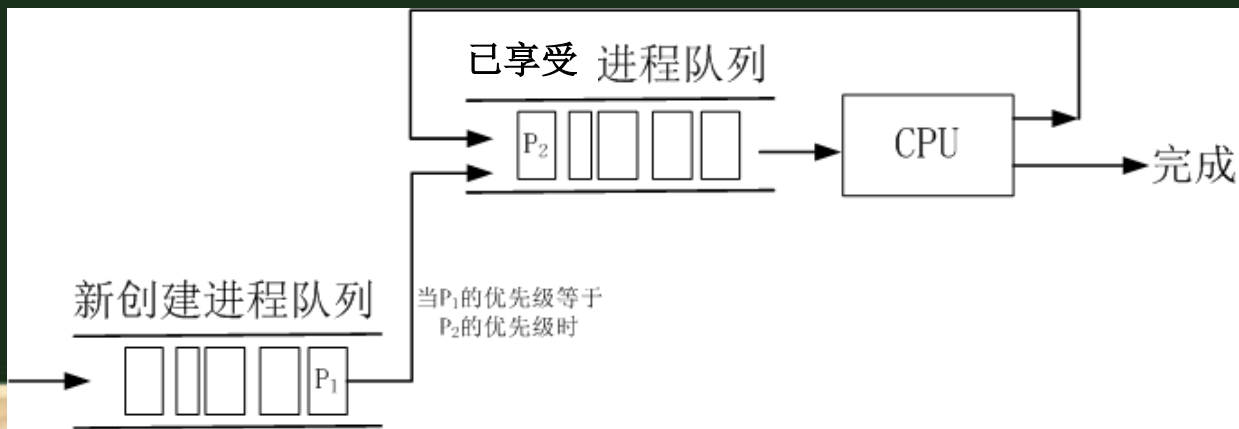
I / O繁忙的进程，CPU繁忙的进程，I / O与CPU均衡的进程，其他进程。

优先级法调度算法---进程动态优先级

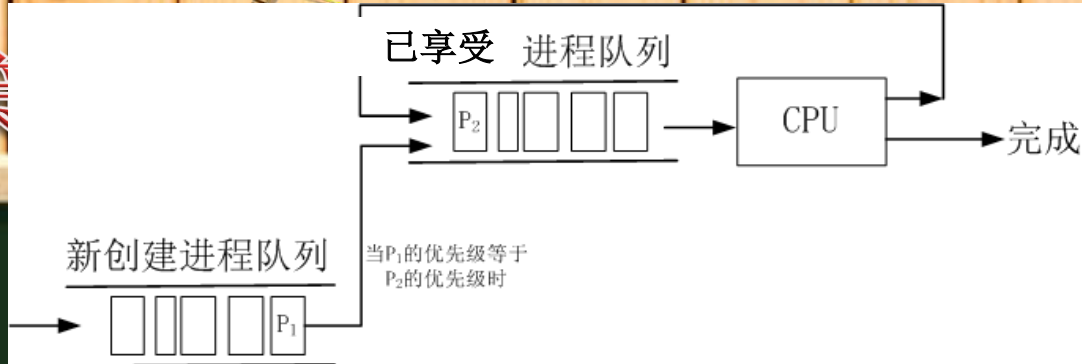
- (1) 根据进程占有CPU时间的长短来决定。一个进程占有处理机的时间愈长，则在被阻塞之后再次获得调度的优先级就越低，反之，其获得调度的可能性就会越大。
- (2) 根据就绪进程等待CPU的时间长短来决定。一个就绪进程在就绪队列中等待的时间越长，则它获得调度选中的优先级就越高。

线性优先级调度策略

➤ 新创建的进程按FCFS方式排成就绪队列，而其他已得到过时间片服务的进程也按FCFS方式排成另一个就绪队列或称享受服务队列

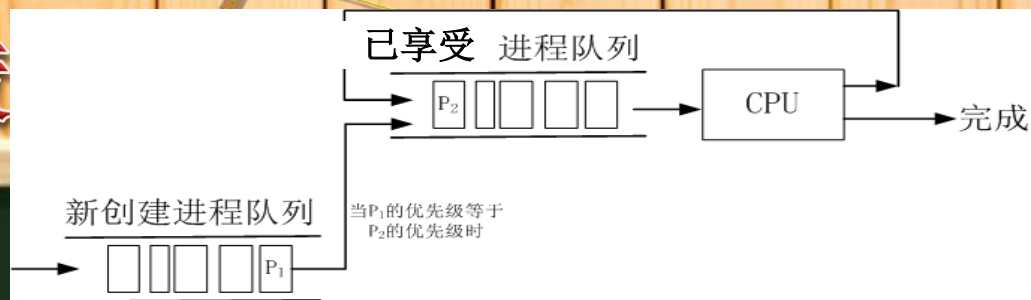


线性优先级调度策



- 对于这两个不同队列中的进程，设新创建进程就绪队列中进程的优先级 P 以 $P=a*t$ ($a>0$) 的速率增加。
- 享受服务队列中进程的优先级 P 以 $P=b*t$ ($a>b>0$) 的速率增长。
- 设某一进程在时刻 t_1 时被创建，在时刻 t 时，该进程的优先级为： $P(t)=a*(t-t_1)$ ($t_1<t<t_1'$)

线性优先级调度策略



➤ 当新创建进程就队列表中的头一个进程的优先级

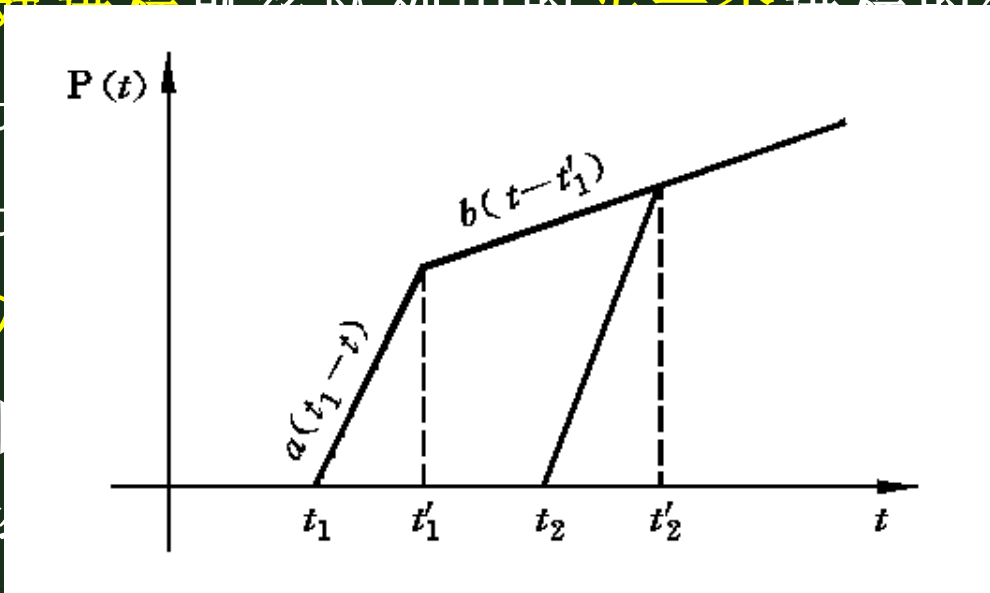
$$P(t) = a * (t - t_1)$$

优先级 $P(t)$

程可以转入

➤ 当享受服

个进程也将

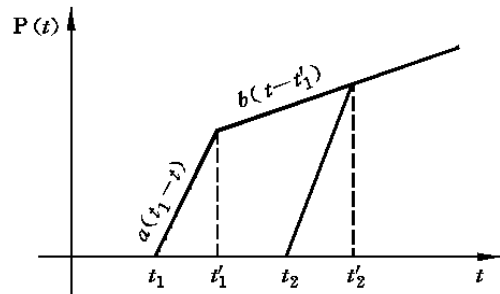


进程的

一个进

的头一

线性优先级调度策略



➤在上述线性优先级调度法中， $a > b > 0$ 的条件是必要的。否则，当 $b > a > 0$ 时，两个不同队列中的就绪态进程的优先级将永远不会相等，从而，在享受服务进程队列中永远只有一个进程。因此，上述线性优先级调度策略退回到FCFS方式。另外，如果 $a > b = 0$ ，则线性优先级调度策略退回到轮转法调度方式。

最短作业优先法 (SJF)

- 最短作业优先法 (SJF) 就是选择那些估计需要执行时间最短的作业投入执行，为它们创建进程和分配资源。直观上来说，采用最短作业优先的调度算法，可使得系统在同一时间内处理的作业个数最多，从而吞吐量也就大于其他调度方式。
- 但是，对于一个不断有作业进入的批处理系统来说，最短作业优先法有可能使得那些长作业永远得不到调度执行的机会。

最高响应比优先法 (HRN)

➤ 是对FCFS方式和SJF 方式的一种综合平衡。HRN调度策略同时考虑每个作业的等待时间长短和估计需要执行的运行时间，从中选出响应比最高的作业投入执行。
T为该作业估计需要的执行时间

➤ 响应比R定义如下： $R = (W + T) / T = 1 + W / T$

W为作业在后备状态队列中的等待时间

最高响应比优先法 (HRN)

➤ 每当要进行作业调度时，系统**计算每个作业的响应比**，选择其中**R最大者投入执行**。这样，即使是长作业，随着它等待时间的增加， W/T 也就随着增加，也就有机会获得调度执行。

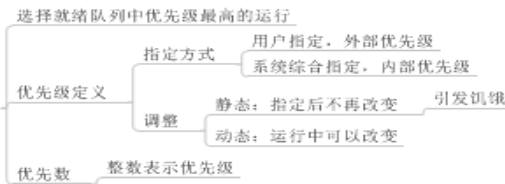
最高响应比优先法（HRN）

➤这种算法是介于FCFS和SJF 之间的一种折中算法。由于长作业也有机会投入运行，在同一时间内处理的作业数显然要少于SJF 法，从而采用HRN方式时其**吞吐量将小于**采用SJF法时的吞吐量。另外，由于每次调度前要计算响应比，**系统开销也要相应增加**。

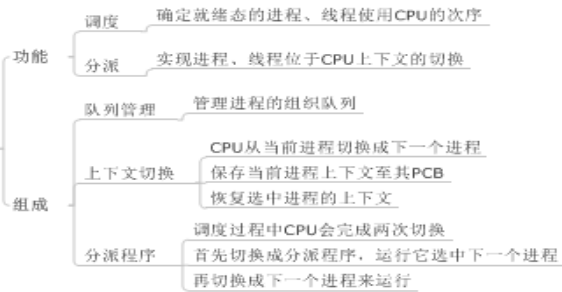
处理器调度算法

可剥夺也可不剥夺

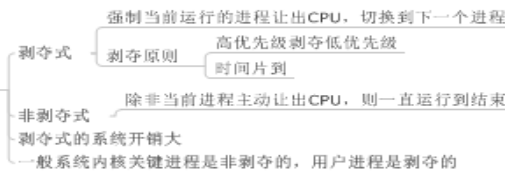
优先级



调度机制



调度的基本分类



实时调度



剥夺算法

SRTF: 最短剩余时间优先

轮转

时间片调度

时间片的确定

思想

多级队列

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

非剥夺算法

FCFS: 先来先服务

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

思想

三种调度策略响应时间评价

- 短作业在系统中的驻留平均时间与长作业的驻留平均时间相同，这对短作业是不利的

FCFS方式

1

- 所需服务时间短的顾客的响应时间小于所需服务时间长的顾客的响应时间。

- 轮转法在响应时间上要优于FCFS调度方式

RR方式

2

- 对于服务时间短的顾客，其响应时间为 $R_{rr} < R_{sr} < R_{fc}$

- 对于服务时间长的顾客，其响应时间为 $R_{fc} < R_{sr} < R_{rr}$

三种对比

3

练习

➤ 假定要在—台处理机上执行右表中所示的作业，且假定这些作业在时刻0以1、2、3、4、5的顺序到达，说明分别使用FCFS、RR（时间片=1）、SJF以及非剥夺优先级调度算法时，这些作业的执行情况。（优先级的高低顺序依次为1到5），同时针对以上每种调度算法，给出平均周转时间和平均带权周转时间。

作业	执行时间	优先级
1	10	3
2	1	1
3	2	3
4	1	4
5	5	2

FCFS:

1	2	3	4	5
---	---	---	---	---

RR:

1	2	3	4	5	1	3	5	1	5	1	5	1	5	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

SJF:

2	4	3	5	1
---	---	---	---	---

优先级:

2	5	1	3	4
---	---	---	---	---

作业	执行时间	优先级
1	10	3
2	1	1
3	2	3
4	1	4
5	5	2

算法	时间类型	P1	P2	P3	P4	P5	平均时间
	运行时间	10	1	2	1	5	3.8
FCFS	周转时间	10	11	13	14	19	13.4
	加权周转时间	1	11	6.5	14	3.8	7.26
RR	周转时间	19	2	7	4	14	9.2
	加权周转时间	1.9	2	3.5	4	2.8	2.84
SJF	周转时间	19	1	4	2	9	7
	加权周转时间	1.9	1	2	2	1.8	1.74
优先级	周转时间	16	1	18	19	6	12
	加权周转时间	1.6	1	9	19	1.2	6.36

O S

1

分级调度

2

作业调度

3

进程调度

4

调度算法及算法评价

5

实时系统调度算法



实时系统

实时系统与其他系统的区别

- 其处理和控制的正确性不仅仅取决于计算的逻辑结果，而且取决于计算和处理结果产生的时间。
- 它所处理的外部任务可分为周期性的与非周期性的两大类

硬实时任务

- 要求系统必须完全满足任务的时限要求。

软实时任务

- 允许系统对任务的时限要求有一定的延迟，其时限要求只是一个相对条件。

周期性

- 只要在周期 T 内完成或开始进行处理

非周期性

- 存在有一个完成或开始进行处理时限

实时系统的特点



实时操作系统需具备的能力

很快的进程或线程切换速度

进程或线程切换速度是实时系统设计的核心。调度算法的设计原则是满足所有硬实时任务的处理时限和尽可能多地满足软实时任务的处理时限。

快速的外部中断响应能力

第七章详细学习

基于优先级的随时抢先式调度策略

- ① 优先级+时间片~~轮转~~调度策略；
- ② 基于优先级的~~非~~抢先式调度策略；
- ③ 基于优先级的固定点抢先式调度策略；
- ④ 基于优先级的随时抢先式调度策略。

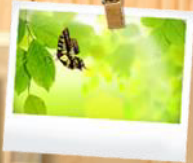
实时调度算法的分类

尽力而
为调度
算法类

静态表
格驱动
类

动态计
划调度
算法类

静态优先
级驱动抢
先式调度
算法类



静态表格驱动类

其主要分析参数为**周期**，**执行时间**、**周期执行结束时限**和**任务优先级**等



- **静态表格驱动类**的实时调度算法，对可能的调度条件和参数进行静态分析，并将分析结果作为实际调度结果。这类调度方法多用于调度处理周期性任务。
- **最早时限优先法**是比较**典型**的**静态表格驱动算法**。这里，最早时限优先法是优先调度时限最早的任务获得处理机的调度方法。

静态表格驱动类—时限优先调度算法

任务就绪时间或事件到达时间



开始时限



完成时限



处理时间



资源需求



优先级

➤ 时限调度算法所需要的相关输入信息包括左面几种：

静态表格驱动类—时限调度算法

- 时限调度算法的基本思想是：按用户的时限要求顺序设置优先级，优先级高者占据处理机，也就是说，**时限要求最近的任务优先占有处理机**。
- 时限调度是**抢先式**的。抢先式时限调度算法必须把新到达任务的时限要求和当前正在执行任务的时限要求进行比较，如果新到达任务的时限要求更近，则应执行新到达的任务。

时限调度算法调度周期性实时任务的例子

进 程	事件发生时限	执行时限	结束时限
<u>DA(1)</u>	0	15	30
<u>DA(2)</u>	30	15	60
<u>DA(3)</u>	60	15	90
...	...	...	...
<u>DB(1)</u>	0	38	75
<u>DB(2)</u>	75	38	150
<u>DB(3)</u>	150	38	225
...	...	...	...

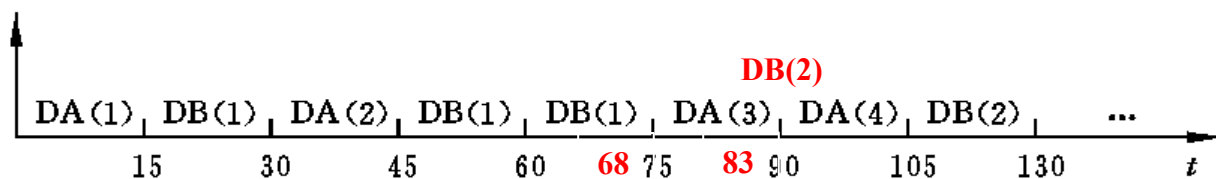


也
为
里
B

时限调度算法调度周期

➤ 如果使用时限调度算法，并按**最近结束时限**优先级最高的方法进行排列，则各进程的调度顺序和相对时间为：

进 程	事件发生时限	执行时限	结束时限
<u>DA(1)</u>	0	15	30
<u>DA(2)</u>	30	15	60
<u>DA(3)</u>	60	15	90
...	...	...	...
<u>DB(1)</u>	0	38	75
<u>DB(2)</u>	75	38	150
<u>DB(3)</u>	150	38	225
...	...	...	...



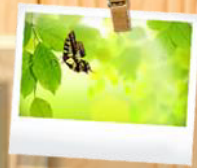
实时调度算法的分类

尽力而
为调度
算法类

静态表
格驱动
类

动态计
划调度
算法类

静态优先
级驱动抢
先式调度
算法类



静态优先级驱动抢先式调度算法类

- 该类算法也进行静态分析，不过，它们的静态分析不直接产生调度结果，而只用来指定任务的优先级。
- **频率单调调度算法**就是一种静态优先级驱动的抢先式调度算法。

静态优先级驱动抢先式调度算法类

--频率单调调度算法

- 频率单调调度算法是一种被广泛用于多周期性实时处理的调度算法。
- 频率单调调度算法的基本原理是频率越低（周期越长）的任务的优先级越低。

静态优先级驱动抢先式调度算法类

--频率单调调度算法

- 设任务周期问题，任务的执行时间为 C ，则使用频率单调调度算法的必要条件是 $C \leq T$ 。
- 已经证明，对于 n ($n \geq 1$) 个周期的不同任务来说，设每个周期为 T_i ，其相应任务的执行时间为 C_i ，则使用频率单调调度算法的充分条件是：

$$\frac{C_1}{T_1} + \frac{C_2}{T_2} + \dots + \frac{C_i}{T_i} + \dots + \frac{C_n}{T_n} \leq n(2^{\frac{1}{n}} - 1)$$

静态优先级驱动抢先式调度算法类

--频率单调调度算法

➤例如，对于由3个周期组成的实时任务序列来说，其执行时间与周期之比应是：

$$\frac{C_1}{T_1} + \frac{C_2}{T_2} + \frac{C_3}{T_3} \leq 3(2^{\frac{1}{3}} - 1) = 0.799$$

➤如果进程执行时间与周期比之和大于 $n(2^{1/n} - 1)$ ，则用户所要求的时限无法保证。

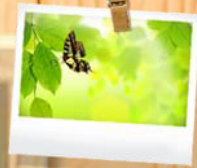
实时调度算法的分类

尽力而
为调度
算法类

静态表
格驱动
类

动态计
划调度
算法类

静态优先
级驱动抢
先式调度
算法类



动态计划调度算法类

➤ 动态计划调度算法在调度任务执行之前排出调度计划，并分析计划的调度结果是否使得任务所要求的处理时限得到满足。如果能够满足，则按调度计划执行，否则修改调度计划。



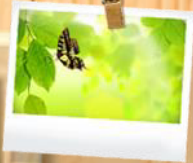
实时调度算法的分类

尽力而
为调度
算法类

静态表
格驱动
类

动态计
划调度
算法类

静态优先
级驱动抢
先式调度
算法类



尽力而为调度算法类

➤ 这一类算法不进行可能性分析，只对到达的事件和相关任务指定相应的优先级，并进行调度。尽力而为调度方式开销较小，实现容易。但是，该算法不一定满足用户要求的处理时限。

作业

➤课后认真复习第四章，下周一进行第三章
+第四章节测试

O S

谢谢大家！

THANK YOU!

